CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

NAAC GRADE A+
Accredited University

Experiment No.: 5

Student Name: Harsh **UID: 24MCA20045**

Branch: MCA **Section/Group: 24MCA-1/B**

Semester: 3rd **Date of Performance: 29/10/25**

Subject Name: Backend Technologies Lab **Subject Code: 24CAP-709**

1. Aim:

- Authenticate the student who is checking marks from university portal as created in experiment.

2. Objective:

- To design a secure login system for student authentication on the university portal.
- To verify student identity using valid credentials (e.g., email/roll number and password).
- To restrict unauthorized access to the marks module.
- To generate and manage authentication tokens or sessions after successful login.
- To ensure data privacy and security of student information.
- To display marks only for authenticated and authorized students.
- To implement proper error handling for invalid or expired logins.

3. Concept to be used:

- **Authentication** – Process of verifying a user's identity before granting access.
- **Authorization** – Ensuring that only verified students can view their own marks.
- **JWT (JSON Web Token) / Session Management** – Used to maintain secure login sessions after authentication.
- **Database Connectivity** – To fetch student details and marks from the database securely.
- **HTTP Methods and Status Codes** – For handling login requests and responses properly.
- **Data Encryption / Password Hashing** – To protect user credentials in the database.

4. Technologies Used:

- **Backend**: Node.js, Express.js
- **Database**: MongoDB with Mongoose
- **Security**: bcryptjs, jsonwebtoken
- **Tools**: Postman for testing APIs

5. Flowchart:

Updated Project Structure

university-course-app/

├── controllers/

│ ├── studentController.js

│ └── authController.js

├── models/

│ └── Student.js

├── routes/

│ ├── studentRoutes.js

│ └── authRoutes.js

6. Source Code:

- **controllers**

➢ **authControllers.js**

```
const Student = require('../models/Student');
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');

exports.login = async (req, res) => {
  const { email, password } = req.body;

  try {
    const student = await Student.findOne({ email });
    if (!student) return res.status(400).json({ msg: 'Invalid credentials' });

    const isMatch = await bcrypt.compare(password, student.password);
    if (!isMatch) return res.status(400).json({ msg: 'Invalid credentials' });
    const token = jwt.sign({ id: student._id }, 'secretKey', { expiresIn: '10m' });
    res.json({ token });
```

```
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
};
```

➢ **studentControllers.js**

```
const Student = require('../models/Student');
exports.getMarks = async (req, res) => {
  try {
    const student = await Student.findById(req.user.id);
    res.json({ name: student.name, marks: student.marks });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
};
```

- **middleware**

➢ **auth.js**

```
const jwt = require('jsonwebtoken');

module.exports = function (req, res, next) {
  const token = req.header('x-auth-token');
  if (!token) return res.status(401).json({ msg: 'No token, access denied' });
  try {
    const decoded = jwt.verify(token, 'secretKey');
    req.user = decoded;
    next();
  } catch (err) {
    res.status(400).json({ msg: 'Token is not valid' });
  }
};
```

- **model.js**

➢ **Student.js**

```
const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');
const StudentSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  password: { type: String, required: true },
  marks: { type: Number, default: 0 },
});
```

```
StudentSchema.pre('save', async function (next) {
  if (!this.isModified('password')) return next();
  this.password = await bcrypt.hash(this.password, 10);
  next();
});
module.exports = mongoose.model('Student', StudentSchema);
```

- **routes**

➢ **authRoutes.js**

```
const express = require('express');
const router = express.Router();
const { login } = require('../controllers/authController');
const Student = require('../models/Student');
// Temporary Registration Route for Testing
router.post('/register', async (req, res) => {
  try {
    const student = new Student(req.body);
    await student.save();
    res.json(student);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});
router.post('/login', login);

module.exports = router;
```

➢ **studentRoutes.js**

```
const express = require('express');
const router = express.Router();
const { getMarks } = require('../controllers/studentController');
const auth = require('../middleware/auth');
router.get('/marks', auth, getMarks);
module.exports = router;
```

- **app.js**

```
const express = require('express');
const mongoose = require('mongoose');
const dotenv = require('dotenv');
dotenv.config();
const authRoutes = require('./routes/authRoutes');
const studentRoutes = require('./routes/studentRoutes');
```

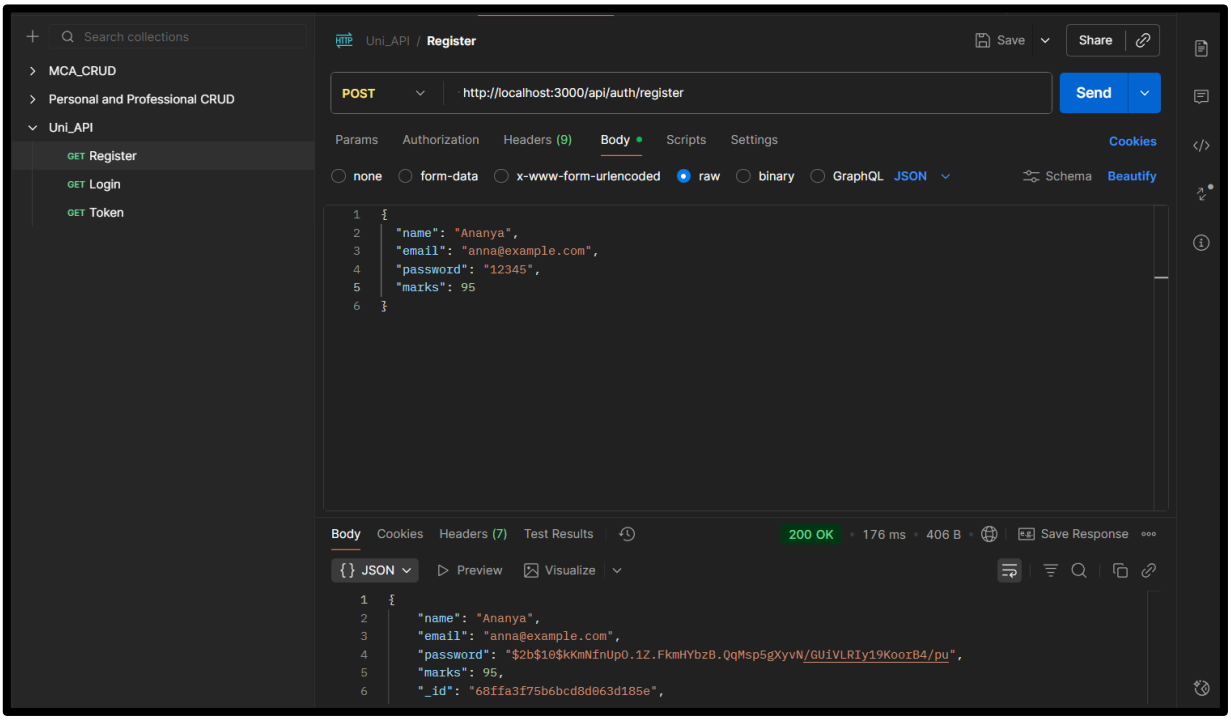
```
const app = express();
app.use(express.json());
// ☑ Connect MongoDB
mongoose.connect('mongodb://127.0.0.1:27017/universitydatabase', {
useNewUrlParser: true,
useUnifiedTopology: true,
})
.then(() => console.log('☑ MongoDB Connected'))
.catch(err => console.error('❌ DB Connection Error:', err));
// ☑ Routes
app.use('/api/auth', authRoutes);
app.use('/api/students', studentRoutes);
// ☑ Start server
const PORT = process.env.PORT || 3000;
app.listen(PORT, () => console.log(`🚀 Server running on port ${PORT}`));
```

- **package.json**

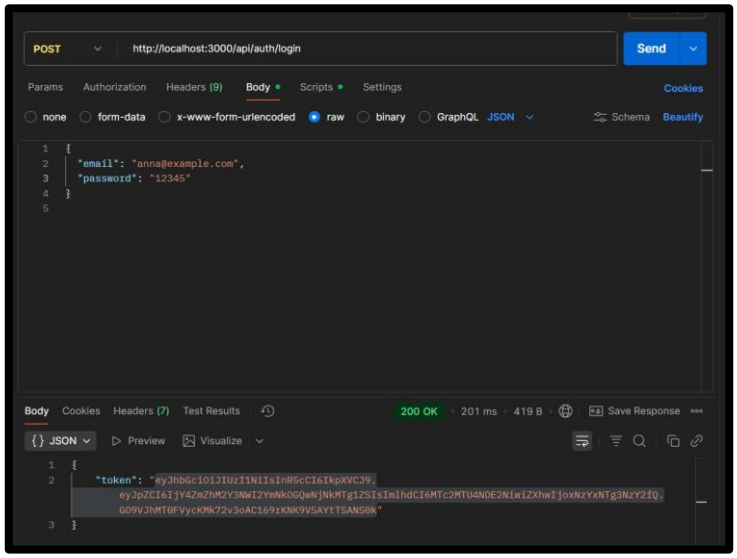
```
{
"name": "uni-course-app",
"version": "1.0.0",
"main": "app.js",
"scripts": {
"test": "echo \"Error: no test specified\" && exit 1"
},
"keywords": [],
"author": "",
"license": "ISC",
"description": "",
"dependencies": {
"bcryptjs": "^3.0.2",
"dotenv": "^17.2.3",
"express": "^5.1.0",
"jsonwebtoken": "^9.0.2",
"mongoose": "^8.19.2"
}
}
```

5. Output:

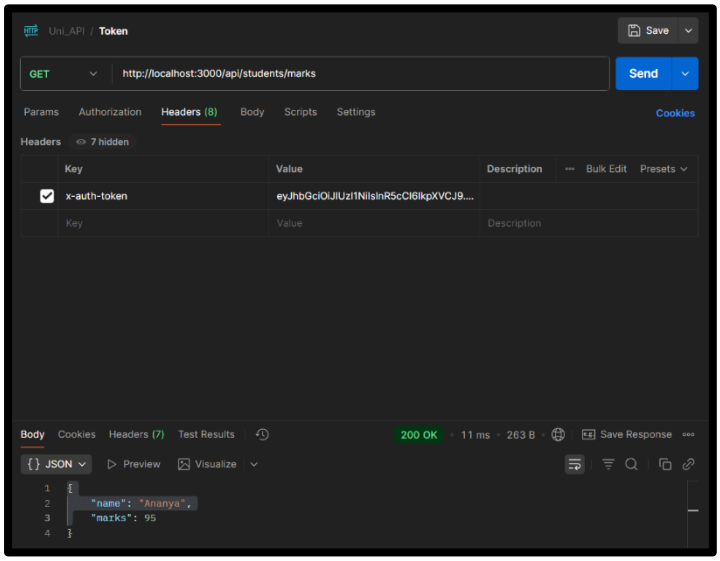
Register:



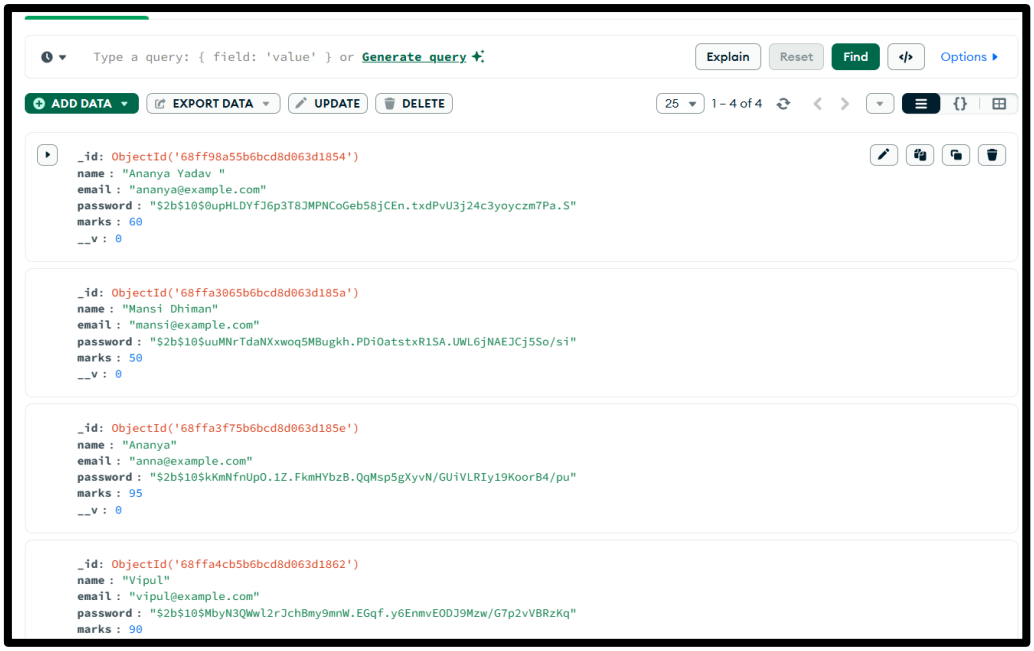
Login(generate Token):



View Marks(using Token Authentication):



MongoDB(UniversityDatabase):



6. Learning Outcomes:

- Learned to implement **student authentication** using **JWT**.
- Understood **password hashing** with **bcrypt.js** for security.
- Gained knowledge of using **middleware** for token verification.
- Learned to create **protected routes** accessible only to authenticated users.